

Dynamic Programming Notes

Maximum Dot Product Problems (LeetCode + GFG)

Problem 1: LeetCode 1458

Maximum Dot Product of Two Subsequences

Problem Statement

Given two integer arrays `nums1` and `nums2`, return the maximum dot product between **non-empty subsequences** of the arrays.

A subsequence of a list is obtained by deleting some elements (possibly none).

The dot product of two sequences of equal length is:

$$a_1 * b_1 + a_2 * b_2 + \dots + a_k * b_k$$

Example

```
nums1 = [2,1,-2,5]
nums2 = [3,0,-6]

Answer = 18
```

Choose:

```
[2,-2]
[3,-6]
```

Dot product:

$$\begin{aligned} &2*3 + (-2)*(-6) \\ &= 6 + 12 \\ &= 18 \end{aligned}$$

Step 1: Identify State

Since there are two arrays:

```
f(i,j)
```

means:

```
Maximum dot product using:
```

```
nums1[i:]  
nums2[j:]
```

Step 2: Identify Choices

At position (i, j) :

Choice 1: Match Both

```
nums1[i] * nums2[j]
```

Continue:

```
nums1[i]*nums2[j] + f(i+1,j+1)
```

Choice 2: Skip from First Array

```
f(i+1,j)
```

Choice 3: Skip from Second Array

```
f(i,j+1)
```

Hidden Constraint

Question says:

Subsequence must be NON-EMPTY.

So this is wrong:

```
if i==m or j==n:  
    return 0
```

because:

```
nums1=[-1]  
nums2=[1]
```

Answer should be:

-1

But recursion returns:

```
max(-1,0)  
=0
```

which is invalid.

Important Pattern

Whenever question says:

- non-empty
- at least one
- mandatory pick

Think:

```
return -inf
```

instead of:

```
return 0
```

Base Case

```
if i==m or j==n:  
    return -inf
```

Why Need **prod** Separately?

Suppose:

```
nums1=[-1]  
nums2=[1]
```

Then:

```
prod=-1  
f(i+1,j+1)=-inf
```

So:

```
prod + f(...)
```

becomes:

```
-inf
```

Wrong.

Therefore:

```
take = max(  
    prod,  
    prod + f(i+1,j+1)  
)
```

This means:

Take current pair and STOP.

OR

Take current pair and CONTINUE.

Recurrence

```
prod = nums1[i]*nums2[j]  
  
take = max(  
    prod,  
    prod + f(i+1,j+1)  
)  
  
skip1 = f(i+1,j)  
  
skip2 = f(i,j+1)  
  
f(i,j) = max(  
    take,  
    skip1,  
    skip2  
)
```

Pure Recursion

```
def solve(i,j):  
  
    if i==m or j==n:  
        return float('-inf')
```

```
prod = a[i]*b[j]

take = max(
    prod,
    prod + solve(i+1,j+1)
)

skip1 = solve(i+1,j)
skip2 = solve(i,j+1)

return max(
    take,
    skip1,
    skip2
)
```

Memoization

```
from functools import lru_cache

@lru_cache(None)
def solve(i,j):

    if i==m or j==n:
        return float('-inf')

    prod = a[i]*b[j]

    take = max(
        prod,
        prod + solve(i+1,j+1)
    )

    skip1 = solve(i+1,j)
    skip2 = solve(i,j+1)

    return max(
        take,
        skip1,
        skip2
    )

return solve(0,0)
```

DP Table Building

State:

```
dp[i][j]
=
answer using:

a[i:]
b[j:]
```

Table size:

```
(m+1) x (n+1)
```

Base Initialization

```
dp[m][*] = -inf
dp[*][n] = -inf
```

because empty subsequence is not allowed.

Filling Direction

Since:

```
dp[i][j]
depends on

dp[i+1][j]
dp[i][j+1]
dp[i+1][j+1]
```

we fill:

```
for i from m-1 to 0:
    for j from n-1 to 0:
```

Tabulation Code

```
def maxDotProduct(a,b):

    m = len(a)
    n = len(b)

    NEG = float('-inf')

    dp = [[NEG]*(n+1)
           for _ in range(m+1)]

    for i in range(m-1,-1,-1):
        for j in range(n-1,-1,-1):

            prod = a[i]*b[j]

            take = max(
                prod,
                prod + dp[i+1][j+1]
            )

            skip1 = dp[i+1][j]
            skip2 = dp[i][j+1]

            dp[i][j] = max(
                take,
                skip1,
                skip2
            )

    return dp[0][0]
```

Complexity

Time:

$O(m*n)$

Space:

$O(m*n)$

Patterns Learned

Pattern 1

Two arrays:

```
f(i,j)
```

Choices:

```
Take both  
Skip first  
Skip second
```

Examples:

- LCS
- Edit Distance
- Distinct Subsequences
- Uncrossed Lines
- Maximum Dot Product

Pattern 2

Question says:

```
non-empty  
at least one  
mandatory pick
```

Think:

```
return -inf
```

Pattern 3

Whenever using `-inf`:

```
take = max(  
    current,
```

```
)      current + recurse
```

Problem 2: GFG Maximum Dot Product

Problem Statement

Given two arrays of lengths m and n .

You may insert zeros into the smaller array so that both arrays become equal length.

Find the maximum dot product.

Example

```
a = [2,3,1,7,8]
b = [3,6,7]
```

Answer:

```
2*0 + 3*3 + 1*0 + 7*6 + 8*7
= 107
```

Key Observation

You are NOT selecting arbitrary subsequences.

You are:

```
Matching every element of smaller array
with some elements of larger array.
```

Skipping means:

```
Insert zero.
```

State

```
f(i,j)
```

means:

```
Maximum answer using:
```

```
big[i:]  
small[j:]
```

Choices

At position (i,j):

Match

```
big[i]*small[j]  
+  
f(i+1,j+1)
```

Skip current element of bigger array

Equivalent to:

```
big[i]*0
```

So:

```
f(i+1,j)
```

Recurrence

```
f(i,j)
=
max(
    big[i]*small[j]
    +
    f(i+1,j+1),
    f(i+1,j)
)
```

Base Cases

If all elements of smaller array are used:

```
if j==n:
    return 0
```

If bigger array finishes first:

```
if i==m:
    return -inf
```

because some elements of smaller array remain unmatched.

Pure Recursion

```
def solve(i,j):

    if j==n:
        return 0

    if i==m:
        return float('-inf')

    take = (
        big[i]*small[j]
        +
        solve(i+1,j+1)
    )

    skip = solve(i+1,j)
```

```
    return max(
        take,
        skip
    )
```

Memoization

```
from functools import lru_cache

@lru_cache(None)
def solve(i,j):

    if j==n:
        return 0

    if i==m:
        return float('-inf')

    take = (
        big[i]*small[j]
        +
        solve(i+1,j+1)
    )

    skip = solve(i+1,j)

    return max(
        take,
        skip
    )

return solve(0,0)
```

DP Table

```
dp[i][j]
=
answer using:

big[i:]
small[j:]
```

Table Size

```
(m+1) x (n+1)
```

Base Initialization

```
for i:  
    dp[i][n]=0  
  
dp[m][j]=-inf
```

Filling Direction

Dependencies:

```
dp[i+1][j]  
dp[i+1][j+1]
```

So:

```
for i from m-1 to 0:  
    for j from n-1 to 0:
```

Tabulation Code

```
def maxDotProduct(a,b):  
  
    if len(a)<len(b):  
        a,b=b,a  
  
    m=len(a)  
    n=len(b)  
  
    NEG=float('-inf')
```

```
dp=[[NEG]*(n+1)
    for _ in range(m+1)]

for i in range(m+1):
    dp[i][n]=0

for i in range(m-1,-1,-1):
    for j in range(n-1,-1,-1):

        take = (
            a[i]*b[j]
            +
            dp[i+1][j+1]
        )

        skip = dp[i+1][j]

        dp[i][j] = max(
            take,
            skip
        )

return dp[0][0]
```

Complexity

Time:

$O(m*n)$

Space:

$O(m*n)$

Pattern Learned

Whenever question says:

Insert zeros
Skip elements of larger array
Match all elements of smaller array

Think:

```
f(i,j)
=
max(
    take,
    skip_big
)
```

NOT:

```
skip first
skip second
```

because every element of the smaller array must be matched.

Final Identification Cheat Sheet

Problem Type	State	Choices
LCS	$f(i,j)$	take, skip1, skip2
Edit Distance	$f(i,j)$	replace, insert, delete
LeetCode 1458	$f(i,j)$	take, skip1, skip2
GFG Maximum Dot Product	$f(i,j)$	take, skip bigger
Distinct Subsequences	$f(i,j)$	take, skip
Uncrossed Lines	$f(i,j)$	take, skip1, skip2

Master Rule

Two Arrays DP

First ask:

```
Can I skip from both arrays?
```

If YES:


```
take  
skip1  
skip2
```

If NO:

Who is mandatory?

Then skip only the optional array.